

[VN] Pod là gì?

[EN] What is a Pod?

[VN] Pod là đơn vị nhỏ nhất có thể deploy trong Kubernetes, đại diện cho một hoặc nhiều container chạy cùng nhau trên cùng một node. Các container trong cùng một Pod chia sẻ network (cùng IP và port space) và có thể chia sẻ volume để dùng chung dữ liệu. Thông thường, một Pod chỉ chứa một container chính, nhưng có thể có thêm sidecar container để hỗ trợ như logging hoặc monitoring. Pod là lớp trừu tượng giúp Kubernetes quản lý, scale và restart container một cách hiệu quả.

[EN] A Pod is the smallest deployable unit in Kubernetes and represents one or more containers running together on the same node. Containers inside the same Pod share the same network (same IP and port space) and can share volumes to access common data. Usually, a Pod contains one main container, but it can also include sidecar containers for tasks like logging or monitoring. A Pod is an abstraction layer that helps Kubernetes manage, scale, and restart containers efficiently.

[VN] Deployment vs StatefulSet.

[EN] Deployment vs StatefulSet.

[VN] Deployment dùng để quản lý các Pod stateless, nghĩa là các Pod có thể thay thế cho nhau và không cần giữ trạng thái riêng. Nó phù hợp cho API server, backend service và hỗ trợ rolling update, rollback dễ dàng. StatefulSet dùng cho ứng dụng stateful như database hoặc message broker, nơi mỗi Pod có identity cố định, hostname cố định và persistent volume riêng. StatefulSet đảm bảo thứ tự tạo, cập nhật và xóa Pod theo thứ tự, đồng thời giữ dữ liệu ổn định khi Pod restart. Tóm lại, Deployment cho stateless service, còn StatefulSet cho stateful application cần lưu trữ ổn định.

[EN] Deployment is used to manage stateless Pods, meaning Pods are interchangeable and do not keep unique state. It is suitable for API servers or backend services and supports easy rolling updates and rollbacks. StatefulSet is designed for stateful applications such as databases or message brokers, where each Pod has a stable identity, fixed hostname, and its own persistent volume. StatefulSet ensures ordered creation, update, and deletion of Pods, and keeps data stable when a Pod restarts. In summary, Deployment is for stateless services, while StatefulSet is for stateful applications that require stable storage.

[VN] Volume vs bind mount?

[EN] Volume vs bind mount?

[VN] Volume và bind mount đều dùng để lưu trữ dữ liệu bên ngoài container, nhưng cách hoạt động khác nhau. Volume được Docker quản lý hoàn toàn và lưu trong thư mục riêng của Docker, giúp quản lý dễ hơn, portable và phù hợp cho production. Bind mount liên kết

trực tiếp một thư mục hoặc file từ host vào container, nên phụ thuộc vào cấu trúc file của máy host và thường dùng cho development. Volume an toàn và ít phụ thuộc môi trường hơn, còn bind mount linh hoạt và tiện khi cần chỉnh sửa code trực tiếp từ host.

[EN] Volume and bind mount are both used to persist data outside a container, but they work differently. A volume is fully managed by Docker and stored in Docker's own directory, making it easier to manage, more portable, and suitable for production. A bind mount directly links a file or directory from the host into the container, so it depends on the host's file structure and is commonly used in development. Volumes are safer and more environment-independent, while bind mounts are more flexible and convenient for editing code directly from the host.

[VN] ENTRYPOINT vs CMD khác nhau thế nào?

[EN] What is the difference between ENTRYPOINT and CMD?

[VN] ENTRYPOINT và CMD đều dùng để định nghĩa lệnh chạy khi container khởi động, nhưng mục đích khác nhau. ENTRYPOINT là lệnh chính, container luôn chạy nó và những gì bạn truyền thêm khi docker run sẽ nối vào sau. CMD là giá trị mặc định hoặc tham số mặc định cho ENTRYPOINT, và có thể dễ dàng bị ghi đè khi bạn chạy container với lệnh khác. Nói ngắn gọn: ENTRYPOINT cố định hành vi chính của container, còn CMD cung cấp tham số hoặc lệnh mặc định có thể thay đổi.

[EN] ENTRYPOINT and CMD both define what runs when a container starts, but they serve different purposes. ENTRYPOINT is the main command, the container always runs it, and anything you pass with docker run is added after it. CMD is the default value or default arguments for ENTRYPOINT, and it can be easily overridden when you run the container with another command. Simply put: ENTRYPOINT fixes the container's main behavior, while CMD provides default or changeable parameters.

[VN] Docker networking gồm những loại nào?

[EN] What types of Docker networking are there?

[VN] Docker networking có một số loại chính. Bridge network là loại mặc định, dùng cho container chạy trên cùng một host và có thể giao tiếp với nhau qua IP nội bộ. Host network cho phép container dùng trực tiếp network của host, giúp giảm overhead nhưng ít cách ly hơn. None network tắt hoàn toàn network của container. Overlay network dùng trong môi trường nhiều host như Docker Swarm, cho phép container trên nhiều máy khác nhau giao tiếp với nhau. Ngoài ra còn có Macvlan network, cho phép container có IP riêng trong cùng mạng với host. Mỗi loại phù hợp với nhu cầu khác nhau về hiệu năng, bảo mật và kiến trúc hệ thống.

[EN] Docker networking includes several main types. Bridge network is the default type, used for containers running on the same host and allows them to communicate via internal IP addresses. Host network lets a container share the host's network directly, reducing overhead but providing less isolation. None network disables networking completely for the container. Overlay network is used in multi-host environments like Docker Swarm, allowing containers on different machines to communicate. There is also Macvlan network, which allows a container to have its own IP address in the same network as the host. Each type fits different needs in terms of performance, security, and system architecture.

[VN] Healthcheck trong Docker dùng để làm gì?

[EN] What is the purpose of Healthcheck in Docker?

[VN] Healthcheck trong Docker dùng để kiểm tra trạng thái hoạt động của container, đảm bảo ứng dụng bên trong vẫn chạy bình thường chứ không chỉ là process còn sống. Docker sẽ chạy một lệnh định kỳ (ví dụ gọi HTTP endpoint hoặc kiểm tra service) và dựa vào kết quả để đánh dấu container là healthy hoặc unhealthy. Nếu container bị unhealthy, hệ thống orchestration như Kubernetes hoặc Docker Compose có thể tự động restart hoặc thay thế container. Healthcheck giúp tăng độ tin cậy và khả năng tự phục hồi của hệ thống.

[EN] Healthcheck in Docker is used to monitor the health status of a container, ensuring that the application inside is working correctly and not just that the process is running. Docker runs a command periodically (for example calling an HTTP endpoint or checking a service), and based on the result, it marks the container as healthy or unhealthy. If the container becomes unhealthy, orchestration systems like Kubernetes or Docker Compose can automatically restart or replace it. Healthcheck improves system reliability and self-healing capability.

[VN] Multi-stage build là gì?

[EN] What is a multi-stage build?

[VN] Multi-stage build là kỹ thuật trong Docker cho phép sử dụng nhiều stage trong một Dockerfile để tách quá trình build và runtime. Ở stage đầu, ta cài đặt đầy đủ dependency và build source code; sau đó ở stage cuối chỉ copy các file cần thiết (binary hoặc artifact) sang một image nhẹ hơn để chạy. Cách này giúp giảm kích thước image, tăng bảo mật vì không chứa tool build, và tối ưu quá trình deploy. Multi-stage build đặc biệt hữu ích cho ứng dụng Go, Node, hoặc Python khi muốn image nhỏ và sạch cho production.

[EN] Multi-stage build is a Docker technique that uses multiple stages in a single Dockerfile to separate the build process from the runtime environment. In the first stage, you install all dependencies and build the source code; in the final stage, you copy only the

necessary files (such as binaries or artifacts) into a smaller runtime image. This approach reduces image size, improves security by excluding build tools, and optimizes deployment. Multi-stage build is especially useful for Go, Node, or Python applications when you want a small and clean production image.

[VN] Service types trong K8s.

[EN] Service types in K8s.

[VN] Trong Kubernetes có một số loại Service chính. ClusterIP là loại mặc định, chỉ cho phép truy cập service bên trong cluster. NodePort mở một port trên mỗi node, cho phép truy cập từ bên ngoài qua IP của node và port đó. LoadBalancer tạo một external load balancer (thường trên cloud) để expose service ra internet. Ngoài ra còn có Headless Service (không có ClusterIP), dùng khi cần truy cập trực tiếp từng Pod, thường dùng với StatefulSet. Mỗi loại Service phù hợp với nhu cầu truy cập nội bộ hoặc bên ngoài khác nhau.

[EN] In Kubernetes, there are several main Service types. ClusterIP is the default type and allows access to the service only inside the cluster. NodePort opens a port on each node, allowing external access through the node's IP and that port. LoadBalancer creates an external load balancer (usually on cloud providers) to expose the service to the internet. There is also Headless Service (without a ClusterIP), used when direct access to individual Pods is needed, commonly with StatefulSet. Each Service type fits different internal or external access requirements.

[VN] Liveness vs Readiness probe.

[EN] Liveness vs Readiness probe.

[VN] Liveness probe dùng để kiểm tra container còn sống hay không. Nếu liveness probe thất bại nhiều lần, Kubernetes sẽ restart container vì cho rằng ứng dụng bị treo hoặc lỗi. Readiness probe dùng để kiểm tra container đã sẵn sàng nhận traffic hay chưa. Nếu readiness probe thất bại, Pod sẽ bị loại khỏi load balancer nhưng không bị restart. Tóm lại, liveness dùng để phát hiện và khôi phục lỗi bằng cách restart, còn readiness dùng để kiểm soát việc nhận request mà không làm gián đoạn hệ thống.

[EN] Liveness probe checks whether a container is still alive. If the liveness probe fails multiple times, Kubernetes restarts the container because it assumes the application is stuck or unhealthy. Readiness probe checks whether the container is ready to receive traffic. If the readiness probe fails, the Pod is removed from the load balancer but not restarted. In summary, liveness is used to detect and recover from failures by restarting the container, while readiness is used to control traffic flow without interrupting the system.

[VN] Resource requests & limits.

[EN] Resource requests & limits.

[VN] Resource requests và limits trong Kubernetes dùng để quản lý tài nguyên CPU và memory cho container. Request là tài nguyên tối thiểu mà container yêu cầu, và Kubernetes sẽ dùng giá trị này để quyết định schedule Pod lên node phù hợp. Limit là mức tối đa container được phép sử dụng; nếu vượt quá memory limit, container có thể bị kill, còn CPU limit sẽ bị throttle. Việc cấu hình đúng request và limit giúp hệ thống ổn định, tránh quá tải node và đảm bảo phân bổ tài nguyên công bằng giữa các Pod.

[EN] Resource requests and limits in Kubernetes are used to manage CPU and memory for containers. A request is the minimum amount of resources a container needs, and Kubernetes uses it to schedule the Pod on a suitable node. A limit is the maximum amount of resources the container can use; if it exceeds the memory limit, it may be killed, and if it exceeds the CPU limit, it will be throttled. Properly configuring requests and limits helps keep the system stable, prevents node overload, and ensures fair resource allocation among Pods.

[VN] Layer caching hoạt động ra sao?

[EN] How does layer caching work?

[VN] Layer caching trong Docker hoạt động dựa trên từng lệnh trong Dockerfile, mỗi lệnh sẽ tạo ra một layer. Khi build image, Docker sẽ kiểm tra xem layer đó đã tồn tại và chưa thay đổi hay chưa; nếu không thay đổi, Docker sẽ dùng lại layer đã cache thay vì build lại từ đầu. Nếu một layer bị thay đổi, tất cả các layer phía sau nó sẽ phải build lại. Vì vậy, nên đặt các lệnh ít thay đổi (ví dụ cài dependency) trước và các lệnh thay đổi thường xuyên (ví dụ copy source code) sau để tận dụng cache tốt hơn. Cách này giúp build nhanh hơn và tiết kiệm tài nguyên.

[EN] Layer caching in Docker works based on each instruction in the Dockerfile, where each instruction creates a layer. During the build process, Docker checks whether a layer already exists and has not changed; if so, it reuses the cached layer instead of rebuilding it. If one layer changes, all layers after it must be rebuilt. Therefore, instructions that change less often (such as installing dependencies) should be placed before frequently changing instructions (such as copying source code) to maximize cache usage. This approach makes the build process faster and more efficient.

[VN] Cách tối ưu Docker image cho production?

[EN] How to optimize Docker image for production?

[VN] Để tối ưu Docker image cho production, tôi sẽ dùng multi-stage build để tách bước build và runtime, chỉ giữ lại file cần thiết trong image cuối. Tôi chọn base image nhẹ như

alpine hoặc distroless để giảm kích thước và giảm lỗ hổng bảo mật. Tôi xóa cache, file tạm và không cài tool build trong image production. Ngoài ra, tôi cấu hình `.dockerignore` để tránh copy file không cần thiết, sắp xếp layer hợp lý để tận dụng cache khi build, và chạy container với user không phải root để tăng bảo mật. Những cách này giúp image nhỏ hơn, build nhanh hơn và an toàn hơn.

[EN] To optimize a Docker image for production, I use multi-stage build to separate build and runtime, keeping only necessary files in the final image. I choose a lightweight base image such as alpine or distroless to reduce size and security risks. I remove cache, temporary files, and avoid installing build tools in the production image. I also configure `.dockerignore` to exclude unnecessary files, organize layers to maximize build cache efficiency, and run the container as a non-root user for better security. These practices make the image smaller, faster to build, and more secure.

[VN] Làm sao giảm size Python Docker image cho production?

[EN] How to reduce the size of a Python Docker image for production?

[VN] Trong các dự án backend, tôi sử dụng Docker để đóng gói và deploy service. Ban đầu tôi dùng Dockerfile khá đơn giản kiểu single-stage build dựa trên image Python chính thức và cài dependency bằng pip hoặc poetry. Cách này chạy được nhưng image khá lớn và thời gian build chậm, đặc biệt trong CI/CD. Sau đó tôi tìm hiểu các best practice và refactor Dockerfile sang multi-stage build, trong đó stage đầu dùng để build dependency và stage cuối chỉ giữ lại các file cần thiết để chạy ứng dụng. Điều này giúp giảm kích thước image vì không mang theo build tools, cache hoặc package không cần thiết. Tôi cũng sử dụng base image nhẹ như python:slim, alpine hoặc distroless để giảm kích thước và giảm attack surface. Ngoài ra tôi cấu hình `.dockerignore` để tránh copy các file không cần thiết như test, git hoặc cache. Khi cài package, tôi dùng `pip install --no-cache-dir` để không lưu cache và xóa các file tạm sau khi cài. Để tối ưu thêm, tôi sắp xếp Docker layer hợp lý và có thể dùng Docker BuildKit cache để tăng tốc cài dependency trong CI. Về bảo mật, tôi chạy container bằng non-root user, pin version image bằng SHA digest để tránh thay đổi ngoài ý muốn, và kiểm tra các dependency có lỗ hổng bảo mật bằng SonarQube scan. Điều này giúp phát hiện sớm các vấn đề bảo mật và đảm bảo image an toàn hơn khi deploy lên môi trường production. Những cách này giúp Docker image nhỏ hơn, build nhanh hơn và an toàn hơn cho môi trường production.

[EN] In my backend projects, I use Docker to package and deploy services. At the beginning, I used a simple Dockerfile with a single-stage build based on the official Python image and installed dependencies using pip or poetry. It worked, but the image size was quite large and the build time was slow, especially in CI/CD pipelines. Later, I learned more best practices and refactored the Dockerfile to use multi-stage builds. In the first

stage, I build dependencies, and in the final stage I only keep the minimal files needed to run the application. This significantly reduces the image size because build tools, caches, and unnecessary packages are not included. I also use lightweight base images such as `python:slim`, `alpine`, or `distroless` to reduce size and attack surface. In addition, I configure `.dockerignore` to exclude unnecessary files like tests, git data, and cache. When installing packages, I use `pip install --no-cache-dir` to avoid storing cache and remove temporary files. To further optimize builds, I organize Docker layers properly and may use Docker BuildKit cache to speed up dependency installation in CI. For security, I run the container as a non-root user, pin the image version using a SHA digest to avoid unexpected changes, and scan dependencies for vulnerabilities using SonarQube scan. This helps detect security issues early and ensures the image is safer for production deployment. These practices help create a Docker image that is smaller, faster to build, and more secure for production environments.

[VN] Làm sao giảm size React Docker image cho production?

[EN] How to reduce the size of a React Docker image for production?

[VN] Để giảm size Docker image cho ứng dụng React trong production, tôi thường dùng multi-stage build. Ở stage đầu, tôi build ứng dụng React bằng Node và cài dependency bằng `pnpm`. Tôi cũng dùng Docker BuildKit cache để cache dependency, giúp build nhanh hơn trong CI/CD. Ở stage cuối, thay vì chạy ứng dụng bằng Node, tôi dùng `nginx` để serve các static file đã build. Để tối ưu hơn, tôi chỉ giữ lại các file build cần thiết và loại bỏ những thành phần không cần thiết. Trong một số trường hợp, tôi còn build `nginx` static binary với các module cần thiết như `SSL` hoặc `HTTP/2` và tắt các module không dùng để giảm kích thước và giảm attack surface. Image cuối có thể dùng `scratch` hoặc một base image rất nhỏ, chỉ chứa `nginx` binary và các static React files. Tôi cũng nén static assets bằng `gzip` hoặc `brotli` trong quá trình build để tăng hiệu năng khi serve. Ngoài ra, tôi chạy container bằng non-root user, pin version image bằng SHA digest, và kiểm tra lỗ hổng dependency bằng SonarQube scan để tăng bảo mật. Những cách này giúp Docker image nhỏ hơn, build nhanh hơn và an toàn hơn cho production.

[EN] To reduce the size of a React Docker image for production, I usually use a multi-stage build. In the first stage, I build the React application using Node and install dependencies with `pnpm`. I also use Docker BuildKit cache so dependencies can be reused between builds, which makes CI builds faster. In the final stage, instead of running the app with Node, I use `nginx` to serve the built static files. To optimize further, I only keep the necessary build files and remove unnecessary components. In some cases, I build a static `nginx` binary with only required modules such as `SSL` or `HTTP/2` and disable unused modules to reduce size and attack surface. The final image can use `scratch` or a very minimal base image, containing only the `nginx` binary and the React static files. I also

compress static assets using gzip or brotli during the build process to improve performance. In addition, I run the container as a non-root user, pin image versions using a SHA digest, and scan dependencies for vulnerabilities using SonarQube scan to improve security. These practices make the Docker image smaller, faster to build, and more secure for production.

[VN] Service discovery trong Kubernetes hoạt động ra sao?

[EN] How does service discovery work in Kubernetes?

[VN] Service discovery trong Kubernetes hoạt động thông qua đối tượng Service và DNS nội bộ của cluster. Khi một Pod được tạo, nó có IP động và có thể thay đổi nếu bị restart. Kubernetes Service sẽ tạo một IP cố định (ClusterIP) và một DNS name cho nhóm Pod phía sau nó. Các Pod khác chỉ cần gọi đến tên Service thay vì gọi trực tiếp IP của Pod. kube-proxy sẽ thực hiện load balancing và route request đến một Pod phù hợp. Kubernetes cũng tự động cập nhật danh sách endpoint khi Pod scale up hoặc down, vì vậy các service luôn tìm thấy nhau mà không cần cấu hình thủ công. Cách này giúp hệ thống linh hoạt, tự động và dễ scale.

[EN] Service discovery in Kubernetes works through the Service object and the internal cluster DNS. When a Pod is created, it has a dynamic IP that can change if it restarts. A Kubernetes Service provides a stable IP (ClusterIP) and a DNS name for a group of Pods behind it. Other Pods call the Service name instead of calling Pod IP directly. kube-proxy handles load balancing and routes traffic to an appropriate Pod. Kubernetes automatically updates the endpoints list when Pods scale up or down, so services can always find each other without manual configuration. This makes the system flexible, automated, and easy to scale.

[VN] Ingress hoạt động ra sao?

[EN] How does Ingress work?

[VN] Ingress trong Kubernetes là một resource dùng để quản lý truy cập HTTP/HTTPS từ bên ngoài vào các Service trong cluster. Ingress hoạt động thông qua Ingress Controller, là thành phần thực tế xử lý traffic (ví dụ Nginx hoặc HAProxy). Người dùng định nghĩa rule trong Ingress như domain, path hoặc TLS, và Ingress Controller sẽ dựa vào đó để route request đến đúng Service phía sau. Ingress hỗ trợ routing theo host và path, SSL termination và có thể tích hợp authentication. Nhờ đó, ta có thể quản lý nhiều service thông qua một entry point duy nhất thay vì tạo nhiều LoadBalancer riêng lẻ.

[EN] Ingress in Kubernetes is a resource used to manage external HTTP/HTTPS access to Services inside the cluster. It works together with an Ingress Controller, which is the actual component that handles traffic (such as Nginx or HAProxy). Users define rules in the

Ingress resource, such as domain, path, or TLS configuration, and the Ingress Controller routes requests to the correct backend Service based on those rules. Ingress supports host-based and path-based routing, SSL termination, and can integrate authentication. This allows multiple services to be managed through a single entry point instead of creating separate LoadBalancers for each service.

[VN] Rolling update trong K8s.

[EN] Rolling update in K8s.

[VN] Rolling update trong Kubernetes là cơ chế cập nhật phiên bản mới của application mà không gây downtime. Khi triển khai version mới qua Deployment, Kubernetes sẽ tạo Pod mới trước, sau đó dần dần giảm số lượng Pod cũ theo cấu hình như maxSurge và maxUnavailable. Cách này đảm bảo luôn có một số Pod đang chạy để phục vụ traffic trong suốt quá trình cập nhật. Nếu có lỗi xảy ra, có thể rollback về version trước đó. Rolling update giúp cập nhật hệ thống an toàn, liên tục và giảm rủi ro gián đoạn dịch vụ.

[EN] Rolling update in Kubernetes is a strategy to deploy a new version of an application without downtime. When a new version is deployed through a Deployment, Kubernetes creates new Pods first and then gradually reduces the old Pods based on settings like maxSurge and maxUnavailable. This ensures that some Pods are always running to handle traffic during the update process. If an issue occurs, you can roll back to the previous version. Rolling update allows safe, continuous deployment with minimal service disruption.

[VN] HPA (Horizontal Pod Autoscaler) hoạt động thế nào?

[EN] How does HPA (Horizontal Pod Autoscaler) work?

[VN] HPA (Horizontal Pod Autoscaler) trong Kubernetes tự động scale số lượng Pod dựa trên metric như CPU, memory hoặc custom metric. HPA liên tục theo dõi metric từ Metrics Server hoặc hệ thống monitoring và so sánh với target đã cấu hình. Nếu giá trị vượt quá ngưỡng, HPA sẽ tăng số lượng replica; nếu thấp hơn trong một khoảng thời gian, nó sẽ giảm replica để tiết kiệm tài nguyên. HPA giúp hệ thống tự động thích ứng với traffic thay đổi, tăng khả năng chịu tải và tối ưu chi phí.

[EN] HPA (Horizontal Pod Autoscaler) in Kubernetes automatically scales the number of Pods based on metrics such as CPU, memory, or custom metrics. HPA continuously monitors metrics from the Metrics Server or monitoring system and compares them with the configured target value. If the metric exceeds the threshold, HPA increases the number of replicas; if it stays lower for a period of time, it reduces replicas to save resources. HPA helps the system adapt to changing traffic, improve scalability, and optimize costs.

[VN] CrashLoopBackOff là gì?

[EN] What is CrashLoopBackOff?

[VN] CrashLoopBackOff là trạng thái trong Kubernetes khi một Pod liên tục bị crash sau khi khởi động và Kubernetes cố gắng restart lại nhiều lần nhưng vẫn thất bại. Sau mỗi lần crash, hệ thống sẽ chờ một khoảng thời gian tăng dần trước khi restart lại, gọi là “backoff”. Nguyên nhân thường do lỗi cấu hình, thiếu biến môi trường, lỗi kết nối database hoặc lỗi trong code. Để xử lý, cần kiểm tra log của Pod, kiểm tra cấu hình và đảm bảo dependency bên ngoài hoạt động đúng. CrashLoopBackOff cho thấy ứng dụng không thể khởi động ổn định.

[EN] CrashLoopBackOff is a status in Kubernetes that occurs when a Pod keeps crashing after starting, and Kubernetes repeatedly tries to restart it but fails. After each crash, the system waits for an increasing delay before restarting again, which is called “backoff.” Common causes include configuration errors, missing environment variables, database connection issues, or bugs in the code. To fix it, you should check the Pod logs, review configurations, and ensure external dependencies are working correctly. CrashLoopBackOff indicates that the application cannot start and run stably.

[VN] Tại sao pod bị OOMKilled?

[EN] Why is a pod OOMKilled?

[VN] Pod bị OOMKilled khi container sử dụng nhiều memory hơn giới hạn (memory limit) đã cấu hình trong Kubernetes. Khi vượt quá limit, Linux kernel sẽ kích hoạt cơ chế Out Of Memory (OOM) và kill container để bảo vệ node khỏi bị hết bộ nhớ hoàn toàn. Nguyên nhân có thể do memory leak trong code, load tăng đột ngột, cấu hình memory limit quá thấp hoặc xử lý dữ liệu lớn trong RAM. Để khắc phục, cần kiểm tra log, theo dõi memory usage, tối ưu code hoặc tăng memory limit phù hợp với workload.

[EN] A Pod is OOMKilled when its container uses more memory than the configured memory limit in Kubernetes. When the limit is exceeded, the Linux kernel triggers the Out Of Memory (OOM) mechanism and kills the container to protect the node from running out of memory. Possible causes include memory leaks in the code, sudden traffic spikes, setting the memory limit too low, or processing large data in RAM. To fix this, you should check logs, monitor memory usage, optimize the code, or adjust the memory limit according to the workload.

[VN] Kubernetes autoscaling dựa vào metric nào?

[EN] Which metrics does Kubernetes autoscaling rely on?

[VN] Kubernetes autoscaling thường dựa trên các metric như CPU utilization, memory usage hoặc custom metrics như số lượng request, độ dài queue hoặc RPS. Với HPA

(Horizontal Pod Autoscaler), metric phổ biến nhất là phần trăm CPU so với giá trị request đã cấu hình. Ngoài ra, có thể dùng custom metrics từ Prometheus hoặc external metrics từ hệ thống bên ngoài như Kafka lag hoặc queue length. Việc chọn metric phù hợp rất quan trọng để đảm bảo scale đúng theo tải thực tế và tránh scale quá mức hoặc thiếu tài nguyên.

[EN] Kubernetes autoscaling typically relies on metrics such as CPU utilization, memory usage, or custom metrics like request count, queue length, or RPS. With HPA (Horizontal Pod Autoscaler), the most common metric is CPU usage percentage compared to the configured resource request. It can also use custom metrics from systems like Prometheus or external metrics such as Kafka lag or queue length. Choosing the right metric is important to ensure scaling matches real workload and avoids over-scaling or resource shortage.

[VN] Làm sao scale down an toàn?

[EN] How to safely scale down?

[VN] Để scale down an toàn, trước tiên cần đảm bảo ứng dụng hỗ trợ graceful shutdown, nghĩa là khi Pod nhận tín hiệu dừng, nó ngừng nhận request mới và xử lý hết request đang chạy trước khi tắt. Trong Kubernetes, có thể cấu hình readiness probe để Pod bị loại khỏi load balancer trước khi terminate, và dùng preStop hook để chờ một khoảng thời gian trước khi đóng hoàn toàn. Ngoài ra, cần đảm bảo không mất dữ liệu đang xử lý, ví dụ hoàn tất message trong queue hoặc commit transaction trước khi shutdown. Cách này giúp tránh lỗi, mất dữ liệu và đảm bảo trải nghiệm người dùng ổn định.

[EN] To scale down safely, the application must support graceful shutdown, meaning when a Pod receives a termination signal, it stops accepting new requests and finishes ongoing ones before shutting down. In Kubernetes, you can use readiness probes to remove the Pod from the load balancer before termination, and use a preStop hook to delay shutdown for a short time. It is also important to ensure no in-progress data is lost, such as finishing queued messages or committing transactions before shutdown. This approach prevents errors, data loss, and ensures a stable user experience.

[VN] Cách implement zero-downtime deployment.

[EN] How to implement zero-downtime deployment.

[VN] Để implement zero-downtime deployment, cần đảm bảo luôn có instance đang phục vụ traffic trong quá trình cập nhật. Có thể dùng rolling update trong Kubernetes để tạo Pod mới trước rồi mới dừng Pod cũ, kết hợp readiness probe để chỉ nhận traffic khi Pod sẵn sàng. Ngoài ra, có thể dùng chiến lược blue-green hoặc canary deployment để chuyển traffic dần sang version mới. Ứng dụng cũng cần hỗ trợ graceful shutdown để xử lý hết

request trước khi tắt. Cách này giúp cập nhật phiên bản mới mà không làm gián đoạn người dùng.

[EN] To implement zero-downtime deployment, you must ensure that some instances are always serving traffic during the update process. You can use rolling updates in Kubernetes to start new Pods before stopping old ones, combined with readiness probes to receive traffic only when Pods are ready. You can also apply blue-green or canary deployment strategies to gradually shift traffic to the new version. The application should support graceful shutdown to finish ongoing requests before stopping. This approach allows deploying new versions without interrupting users.

[VN] Làm sao debug container production?

[EN] How to debug a production container?

[VN] Khi debug container production, trước tiên tôi sẽ kiểm tra log bằng lệnh docker logs hoặc thông qua hệ thống logging tập trung. Sau đó, tôi kiểm tra trạng thái container, resource usage (CPU, memory) và healthcheck. Nếu cần, tôi có thể dùng docker exec để vào container kiểm tra file, biến môi trường hoặc kết nối network, nhưng nên hạn chế thay đổi trực tiếp trong production. Tôi cũng kiểm tra cấu hình image, version và environment variable để phát hiện sai khác giữa môi trường. Trong hệ thống dùng Kubernetes, có thể dùng kubectl logs và kubectl describe để xem chi tiết pod và event. Quan trọng là phải có logging, monitoring và tracing từ trước để việc debug hiệu quả và an toàn.

[EN] When debugging a production container, I first check logs using docker logs or a centralized logging system. Then I verify the container status, resource usage (CPU, memory), and healthcheck results. If necessary, I use docker exec to enter the container and inspect files, environment variables, or network connections, but I avoid making direct changes in production. I also review image configuration, version, and environment variables to detect differences between environments. In Kubernetes systems, I use kubectl logs and kubectl describe to inspect pod details and events. It is important to have logging, monitoring, and tracing in place to debug effectively and safely.

[VN] Làm sao monitor cluster?

[EN] How to monitor a cluster?

[VN] Để monitor Kubernetes cluster, cần theo dõi cả hạ tầng và ứng dụng. Thông thường, tôi dùng Prometheus để thu thập metric như CPU, memory, network và trạng thái Pod, kết hợp Grafana để visualize dashboard. Ngoài ra, tôi theo dõi log tập trung bằng hệ thống như ELK hoặc Loki để debug. Cần cấu hình alert khi vượt ngưỡng quan trọng như CPU cao, Pod crash hoặc node không sẵn sàng. Việc monitor nên bao gồm cluster level (node,

control plane) và application level (request latency, error rate) để đảm bảo hệ thống ổn định và phản ứng kịp thời khi có sự cố.

[EN] To monitor a Kubernetes cluster, you need to observe both infrastructure and applications. I typically use Prometheus to collect metrics such as CPU, memory, network usage, and Pod status, combined with Grafana for visualization dashboards. I also use centralized logging systems like ELK or Loki for debugging logs. Alerts should be configured for critical thresholds such as high CPU, Pod crashes, or node not ready status. Monitoring should cover both cluster-level (nodes, control plane) and application-level metrics (request latency, error rate) to ensure system stability and quick response to issues.

VI. Containerization & Orchestration

Pod là gì?

Deployment vs StatefulSet.

Volume vs bind mount?

ENTRYPOINT vs CMD khác nhau thế nào?

Docker networking gồm những loại nào?

Healthcheck trong Docker dùng để làm gì?

Multi-stage build là gì?

Service types trong K8s.

Liveness vs Readiness probe.

Resource requests & limits.

Layer caching hoạt động ra sao?

Cách tối ưu Docker image cho production?

Làm sao giảm size Python Docker image cho production?

Làm sao giảm size React Docker image cho production?

Service discovery trong Kubernetes hoạt động ra sao?

Ingress hoạt động ra sao?

Rolling update trong K8s.

HPA (Horizontal Pod Autoscaler) hoạt động thế nào?

CrashLoopBackOff là gì?

Tại sao pod bị OOMKilled?

Kubernetes autoscaling dựa vào metric nào?

Làm sao scale down an toàn?

Cách implement zero-downtime deployment.

Làm sao debug container production?

Làm sao monitor cluster?